COMPLETE CURRICULUM & PLAIN-ENGLISH COMMAND REFERENCE

Docker Masterclass: From Zero to Production

A structured, beginner-friendly syllabus covering container architecture, images, live container management, persistent volumes, networking, Dockerfiles, Docker Compose, security hardening, and every essential CLI command explained in simple words.

Curriculum Overview: How to Master Docker Step by Step

Think of Docker like a standardized shipping container for software. Before Docker, moving an application from a developer's laptop to a production server often caused crashes because of missing libraries, different operating system versions, or conflicting settings. Docker solves this by bundling your application code together with the exact libraries, runtime, and settings it needs inside an isolated package called a **container**.

This masterclass is organized into **10 progressive modules**. You will start by understanding how containers work under the hood, practice running pre-built tools, learn how to store database files safely, connect multiple services together, write your own **`Dockerfile`** blueprints, and orchestrate full-stack apps with **`Docker Compose`**.

Module 1: Core Concepts — How Containers Work Under the Hood

Learning Objective: Understand why containers replaced traditional virtual machines for modern application deployment and learn how the three building blocks of Docker fit together.

What You Will Learn

- **Containers vs. Virtual Machines (VMs):** A Virtual Machine emulates an entire computer—including a heavy guest operating system—consuming gigabytes of RAM and taking minutes to boot. A Docker container shares your computer's existing Linux kernel, using only megabytes of memory and starting in milliseconds.
- **The Three Core Building Blocks:**
  - **Dockerfile (The Recipe):** A simple text file listing step-by-step instructions to assemble your environment.
  - **Image (The Blueprint / Snapshot):** A read-only package built from a Dockerfile containing your code and dependencies. You can share images on Docker Hub just like sharing code on GitHub.
  - **Container (The Running App):** A live, isolated process launched from an Image. You can spin up one—or one hundred—identical containers from a single Image.
- **Docker Engine Architecture:** How the command-line tool (**`docker`** CLI) sends instructions to the background engine (**`dockerd`** daemon) to build, run, and monitor containers.

Setup & Verification Commands

Command	What It Does in Simple Words
`docker --version`	Prints the installed Docker version number so you know the CLI is ready.
`docker version`	Shows detailed version info for both the client (your terminal) and the server (the background Docker engine).
`docker info`	Displays a complete health and resource summary of your Docker installation, including how many containers are running, total images, CPU count, and total memory available.
`docker run hello-world`	The classic first test: downloads a tiny test image from Docker Hub, starts a container, prints a confirmation message, and exits.
`docker help`	Lists all top-level Docker commands. You can also append `--help` to any command (for example, `docker run --help`) to see all its flags.

Module 2: Working with Docker Images — The Blueprints

Learning Objective: Discover, download, inspect, tag, export, and delete Docker images efficiently while understanding how layered storage saves disk space.

What You Will Learn

- **Docker Hub & Official Images:** Finding trusted pre-built images for databases (`postgres`, `mysql`, `redis`), web servers (`nginx`, `caddy`), and programming languages (`python`, `node`, `golang`).
- **Understanding Tags & Flavors:** Why an image name looks like `python:3.12-slim` or `node:20-alpine`. The part after the colon (`:`) is the version tag. Avoid using `:latest` in real projects so sudden updates do not break your app. Use `-slim` or `-alpine` tags for tiny, fast downloads.
- **Union File System & Layers:** Every Docker image is a stack of read-only layers. If five different apps use `ubuntu:24.04` as their base, Docker stores that base layer only once on your hard drive.

Image Management Commands

Command	What It Does in Simple Words
`docker search <keyword>`	Searches Docker Hub directly from your terminal for public images matching your keyword (for example, `docker search nginx`).
`docker pull <image>:<tag>`	Downloads an image from Docker Hub to your computer without starting a container yet (for example, `docker pull redis:7-alpine`).

Command	What It Does in Simple Words
`docker images` `docker image ls`	Lists every image saved on your computer along with its version tag, unique Image ID, creation date, and file size.
`docker history <image>`	Shows every layer inside an image, the exact instruction that created each layer, and how many megabytes each step added.
`docker image inspect <image>`	Prints the complete technical blueprint of an image in JSON format (default environment variables, exposed ports, architecture, and layer hashes).
`docker tag <src> <target>`	Creates a new alias or version tag pointing to an existing image (for example, `docker tag myapp:1.0 myusername/myapp:latest`).
`docker rmi <image>`	Deletes a downloaded image from your computer to free up disk space. Add `-f` to force removal if needed.
`docker image prune`	Cleans up "dangling" images—leftover untagged layers from old builds. Add `-a` (`docker image prune -a`) to delete all images not currently used by any container.
`docker save -o <file.tar> <image>`	Packs a Docker image into a portable `.tar` file so you can copy it via USB or SCP to a server without internet access.
`docker load -i <file.tar>`	Unpacks and imports a saved `.tar` archive back into your local Docker image library.

Module 3: Running & Managing Containers — Daily Operations

Learning Objective: Master the full lifecycle of a container—starting services in the background, opening interactive shells, mapping ports, passing environment variables, and cleaning up stopped containers.

What You Will Learn

- **Detached vs. Interactive Mode:** Web servers and databases should run quietly in the background using `-d` (detached mode). When you want to explore a Linux container with a keyboard, combine `-i` (keep input open) and `-t` (attach a terminal) as `-it`.
- **Port Publishing (`-p HOST:CONTAINER`):** By default, a container is sealed off from your network. Passing `-p 8080:80` tells Docker: "Whenever someone visits port `8080` on my laptop, forward that traffic to port `80` inside the container."
- **Environment Configuration:** Passing database credentials, API keys, and feature flags into containers at launch time using `-e KEY=VALUE` or `--env-file .env` so you never hardcode secrets inside images.
- **Self-Healing Restart Policies:** Using `--restart unless-stopped` or `--restart on-failure` so Docker automatically revives your app if it crashes or if the host server reboots.

Container Lifecycle & Execution Commands

Command	What It Does in Simple Words
`docker run <image>`	Creates a brand-new container from the specified image (downloading it first if missing) and starts it immediately.
`docker run -d --name <name> <image>`	Starts a container in the background (`-d`) and assigns it a memorable name (`--name web`) instead of a random name.
`docker run -p <host>:<cont> <image>`	Bridges a port on your computer (left side) to a port inside the container (right side)—for example, `-p 3000:80`.
`docker run -it <image> sh`	Launches a container and drops you straight into an interactive command prompt inside it so you can run Linux commands directly.
`docker run --rm <image>`	Runs a container and automatically deletes it the instant it exits—ideal for short scripts, tests, or one-time tools.
`docker run -e KEY=VAL <image>`	Injects an environment variable into the container (or use `--env-file .env` to load many variables from a file at once).
`docker run --restart unless-stopped <img>`	Configures Docker to automatically restart the container if the app crashes or when your computer starts up.
`docker ps`	Shows a table of all containers that are currently running, their IDs, names, uptime, and open ports.
`docker ps -a`	Lists every container on your machine—both currently running ones and ones that have stopped or exited in the past.
`docker stop <container>`	Gracefully stops a running container by sending a polite shutdown signal (`SIGTERM`), giving the app 10 seconds to save work before stopping.
`docker start <container>`	Turns an existing stopped container back on without losing any settings or files inside it.
`docker restart <container>`	Stops and immediately starts a container again—handy when you want an app to reload its configuration.
`docker kill <container>`	Forces an unresponsive container to shut down immediately without waiting for cleanup (`SIGKILL`).
`docker pause <cont>` `docker unpause <cont>`	Freezes all running processes inside a container in RAM without stopping it, and unfreezes them when you are ready.
`docker rm <container>`	Permanently deletes a stopped container. Use `docker rm -f <container>` to force-delete a container even if it is still running.

Command	What It Does in Simple Words
`docker container prune`	Deletes all stopped containers in one sweep so your `docker ps -a` list stays clean.
`docker rename <old> <new>`	Changes the friendly name of an existing container without restarting it.

Module 4: Debugging, Logs & Live Container Inspection

Learning Objective: Diagnose why a container crashed, inspect live logs, open a shell inside an already-running service, transfer files, and monitor CPU/RAM usage.

What You Will Learn

- **Why Containers Exit Immediately:** A Docker container stays alive only as long as its main foreground process (PID 1) is running. If your web server crashes or finishes its script, the container stops. Reading `docker logs` is always your first debugging step.
- **`docker run -it` vs. `docker exec -it`:** Beginners often confuse these two! Remember: `docker run` creates a *brand-new* container, whereas `docker exec` steps inside a container that is *already running*.
- **Resource Guardrails:** Preventing a single leaky container from freezing your computer by setting memory limits (`--memory="512m"`) and CPU caps (`--cpus="1.5"`).

Debugging, Monitoring & File Transfer Commands

Command	What It Does in Simple Words
`docker logs <container>`	Prints all console output and error messages that the container has generated since it started.
`docker logs -f --tail 100 <cont>`	Shows the most recent 100 log lines and stays open to stream new log messages live in real time (press `Ctrl+C` to stop).
`docker exec -it <cont> bash`	Opens a live interactive terminal inside a running container so you can inspect folders, check config files, or run CLI tools (use `sh` if `bash` isn't installed).
`docker exec <cont> <command>`	Runs a single command inside a running container and prints the result right in your host terminal (for example, `docker exec db pg_isready`).
`docker cp <src> <cont>:<dest>`	Copies files or folders between your computer and a container in either direction (for example, `docker cp ./config.json web:/etc/app/`).

Command	What It Does in Simple Words
`docker stats`	Opens a live task-manager style dashboard showing real-time CPU%, RAM usage, network traffic, and disk activity for every running container.
`docker top <container>`	Lists all active Linux processes running inside a container without needing to open a shell inside it.
`docker inspect <container>`	Outputs full low-level metadata about a container, including its internal IP address, mounted volume paths, exit code, and health status.
`docker port <container>`	Quickly displays which ports on your host computer are mapped to which ports inside the specified container.
`docker diff <container>`	Shows every file inside the container that was Added (`A`), Changed (`C`), or Deleted (`D`) since the container started.
`docker update --memory 1g <cont>`	Updates CPU or RAM limits on a live container immediately without needing to recreate it.
`docker export -o app.tar <cont>`	Snapshots a container's entire current filesystem into a flat `.tar` archive file.

Module 5: Persistent Storage — Volumes & Bind Mounts

Learning Objective: Store database records, uploaded files, and live source code outside the temporary container layer so your data is never lost when containers are deleted or upgraded.

What You Will Learn

- **The Golden Rule of Container Storage:** Containers are meant to be disposable. Anything saved inside a container's default writable layer is permanently erased when you run `docker rm`.
- **Named Volumes (Best for Databases & Production Data):** Created and managed by Docker in a dedicated area of your host disk. Even if you delete your PostgreSQL container and replace it with a newer version, attaching the same named volume preserves all your tables and rows intact.
- **Bind Mounts (Best for Local Code Development):** Directly maps a folder on your laptop (like `/Users/me/project/src`) into a folder inside the container (`/app/src`). Every time you press Save in VS Code, the container sees the updated code immediately.

Storage & Volume Commands

Command	What It Does in Simple Words
`docker volume create <name>`	Creates a dedicated, persistent storage bucket managed by Docker (for example, `docker volume create pgdata`).

Command	What It Does in Simple Words
`docker volume ls`	Lists all named volumes currently stored on your computer.
`docker volume inspect <name>`	Displays where the volume lives on your host disk and when it was created.
`docker run -v <vol>:<path> <img>`	Mounts a named volume into a folder inside the container (for example, `-v pgdata:/var/lib/postgresql/data`) so database files survive container deletion.
`docker run -v $(pwd):/app <img>`	Bind-mounts your current terminal folder directly into `/app` inside the container so local file edits sync live in both directions.
`docker run -v $(pwd):/app:ro <img>`	Mounts your local folder in **read-only** mode (`:ro`) so the container can read your files but cannot modify or delete them.
`docker run --mount type=volume,src=v,dst=/d`	The clearer, verbose syntax for mounting volumes, bind mounts (`type=bind`), or RAM-only disks (`type=tmpfs`).
`docker volume rm <name>`	Permanently deletes a named volume and all the files inside it (only works if no container is using it).
`docker volume prune`	Deletes all unused volumes that are not currently attached to any container. Use with caution—this permanently erases their stored data!

Module 6: Docker Networking — Connecting Containers Together

Learning Objective: Create isolated virtual networks so your web backend can talk to your database and cache using simple container names as hostnames.

What You Will Learn

- **Why Custom Bridge Networks Are Magic:** When you create your own network (e.g., `docker network create app-net`) and attach both your backend container (named `api`) and database container (named `db`), Docker runs an internal DNS server. Your API can connect to `postgres://db:5432` by name—no hardcoded IP addresses required!
- **Comparing Network Drivers:**
  - **Bridge:** The standard private virtual network for containers on a single machine.
  - **Host:** Shares your computer's network stack directly with zero isolation.
  - **None:** Completely cuts off internet and local network access for high-security offline tasks.

Networking Commands

Command	What It Does in Simple Words
`docker network ls`	Lists all virtual networks on your Docker host, including default networks (**`bridge`**, **`host`**, **`none`**) and your custom ones.
`docker network create <name>`	Creates a new private virtual bridge network so containers attached to it can find each other by name.
`docker run --network <net> <img>`	Launches a container and plugs it directly into the specified virtual network at startup.
`docker network connect <net> <cont>`	Connects an already-running container to an additional network on the fly (a single container can belong to multiple networks).
`docker network disconnect <net> <c>`	Unplugs a running container from the specified network.
`docker network inspect <name>`	Shows detailed network configuration, subnet ranges, and the exact IP address of every container currently connected to that network.
`docker network rm <name>`	Deletes a custom virtual network once no containers are connected to it.
`docker network prune`	Removes all unused custom networks that have zero active containers attached.

Module 7: Writing Dockerfiles & Building Custom Images

Learning Objective: Write clean, fast-building, production-grade **`Dockerfile`** recipes to package your own Python, Node.js, Go, or Java applications.

What You Will Learn

- **Smart Layer Caching:** Docker caches every step of a Dockerfile. If you copy **`package.json`** and run **`npm install`** *before* copying the rest of your source code (**`COPY . .`**), Docker skips reinstalling packages whenever you only edit code—cutting build times from minutes to under a second.
- **Using `.dockerignore`:** Just like **`.gitignore`**, a **`.dockerignore`** file stops Docker from copying heavy or sensitive folders like **`node_modules`**, **`.git`**, **`venv`**, and **`.env`** into your image.
- **Multi-Stage Builds:** Use one stage with compilers and build tools to compile your app, then start a second stage from a tiny runtime image and copy *only* the compiled output across—shrinking 1 GB images down to 30 MB.

Complete Dockerfile Instructions Reference

Instruction	What It Does in Simple Words
`FROM <image>:<tag>`	Selects the starting base image for your project (for example, `FROM python:3.12-slim`). Every Dockerfile begins here.
`WORKDIR /app`	Creates a folder inside the image and sets it as the active working directory for all subsequent commands (like `cd /app`).
`COPY <src> <dest>`	Copies files or folders from your computer into the image filesystem (for example, `COPY requirements.txt ./`).
`ADD <src> <dest>`	Similar to `COPY`, but automatically unpacks local compressed `.tar.gz` archives. Always prefer `COPY` for normal files.
`RUN <command>`	Executes a terminal command while building the image—used to install system tools or language packages (for example, `RUN pip install -r requirements.txt`).
`ENV KEY="value"`	Sets a permanent environment variable inside the image that will also be present when containers run (for example, `ENV NODE_ENV=production`).
`ARG KEY="default"`	Defines a temporary variable used *only* during the build process (not saved in the final running container).
`EXPOSE <port>`	Documents which internal port the application listens on (for example, `EXPOSE 8000`). You still use `-p` when running the container to open it on the host.
`USER <username>`	Switches away from the administrator (`root`) account to a safer, restricted user account before starting the app.
`HEALTHCHECK CMD <cmd>`	Gives Docker a command (like `curl -f http://localhost:8000/health`) to run periodically to check if your app is still healthy.
`ENTRYPOINT ["exec"]`	Locks the container to run a specific main program; any extra arguments passed to `docker run` get appended to it as flags.
`CMD ["exec", "arg1"]`	Specifies the default startup command that launches your app when a container starts (for example, `CMD ["node", "server.js"]`).

Image Building & Registry Publishing Commands

Command	What It Does in Simple Words
`docker build -t <name>:<tag> .`	Reads the **`Dockerfile`** in the current folder (notice the trailing dot **`.`**) and builds an image named **`<name>:<tag>`**.
`docker build -f <file> -t <n> .`	Builds an image using a custom Dockerfile filename (for example, **`-f Dockerfile.dev`**).
`docker build --no-cache -t <n> .`	Forces Docker to execute every step fresh from scratch, ignoring any previously cached build layers.
`docker build --build-arg K=V -t <n> .`	Passes a custom value into an **`ARG`** variable declared inside your Dockerfile at build time.
`docker login`	Authenticates your terminal with Docker Hub (or another registry like GitHub Container Registry) so you can upload images.
`docker push <user>/<img>:<tag>`	Uploads your locally built image to the cloud registry so teammates or production servers can download it.
`docker logout`	Logs out and removes stored registry credentials from your computer.

Module 8: Docker Compose — Orchestrating Multi-Container Apps

Learning Objective: Replace complex multi-step shell scripts with a single **`compose.yaml`** file that launches your frontend, backend API, database, and cache together with one command.

What You Will Learn

- **Why Docker Compose Is Essential:** Real applications rarely run in a single container. You usually have a web app, a database, a background worker, and a Redis cache. **`compose.yaml`** lets you define all of their ports, volumes, networks, and environment variables in one clean YAML file.
- **Automatic Networking & Startup Order:** Compose automatically creates a shared private network for all services in your file and lets you use **`depends_on`** with healthchecks so your backend waits until the database is ready before starting.

Docker Compose Commands

Command	What It Does in Simple Words
`docker compose up`	Reads **`compose.yaml`**, creates the network and volumes, starts all containers in the stack, and shows combined live logs on your screen.
`docker compose up -d`	Starts the entire multi-container application stack quietly in the background and returns control of your terminal.

Command	What It Does in Simple Words
`docker compose up --build -d`	Rebuilds any custom Dockerfiles in your project first so recent code changes are included, then starts the stack in the background.
`docker compose down`	Stops all containers in the project and removes the containers and internal network, while safely keeping your named database volumes intact.
`docker compose down -v`	Stops and removes everything **including named volumes**—use this when you want to wipe the database clean and start fresh.
`docker compose ps`	Lists the status, health, and port mappings of the containers belonging to your current Compose project.
`docker compose logs -f <svc>`	Streams live logs for one specific service (e.g., `docker compose logs -f api`), or omit the service name to watch all services together.
`docker compose exec <svc> sh`	Opens an interactive shell inside a running service container using its short service name from `compose.yaml`.
`docker compose run --rm <svc> <cmd>`	Spins up a temporary one-off container for a service to run a task (such as database migrations or unit tests) and deletes it when finished.
`docker compose stop` `docker compose start`	Pauses or resumes the project's containers without deleting and recreating them.
`docker compose restart <svc>`	Restarts all services in the stack or just the specific service you name.
`docker compose build`	Builds or rebuilds images for services that have a `build:` section in `compose.yaml` without starting containers.
`docker compose pull`	Downloads the newest versions of any pre-built registry images referenced in your `compose.yaml`.
`docker compose config`	Checks your `compose.yaml` file for syntax errors and prints the final resolved configuration with all `.env` variables substituted.

Module 9: System Maintenance, Security & Production Best Practices

Learning Objective: Keep your disk clean, harden containers against security threats, and build cross-platform images that run smoothly on both ARM laptops and Intel/AMD cloud servers.

Production Security Checklist

- **Run as a Non-Root User:** By default, processes inside a container run as **`root`**. Always add a **`USER`** instruction in your Dockerfile so an attacker who exploits a web vulnerability cannot gain root privileges.
- **Keep Images Minimal:** Fewer installed packages mean a smaller attack surface and faster deployments. Prefer **`alpine`**, **`-slim`**, or Google's **`distroless`** base images over full operating system images.
- **Never Bake Secrets into Images:** Never put API keys, SSH private keys, or **`.env`** files into a **`COPY`** or **`ENV`** instruction in a Dockerfile—anyone who pulls the image can read its history. Pass secrets at runtime instead.

System Cleanup & Multi-Architecture Commands

Command	What It Does in Simple Words
`docker system df`	Shows a breakdown of how many gigabytes of disk space are currently used by Docker images, containers, volumes, and build cache.
`docker system prune`	The safe all-in-one cleanup command: removes all stopped containers, unused networks, dangling images, and old build cache.
`docker system prune -a --volumes`	Deep factory-style cleanup: deletes **every** image not used by a running container plus all unused volumes to reclaim maximum disk space.
`docker builder prune`	Clears cached build layers from past **`docker build`** runs while leaving your downloaded and built images untouched.
`docker events`	Streams live real-time notifications from the Docker engine whenever containers start, stop, fail healthchecks, or mount volumes.
`docker context ls`	Lists local and remote Docker environments so you can point your local terminal at a remote cloud server over SSH.
`docker buildx build --platform linux/amd64,linux/arm64 -t <img> --push .`	Builds a universal multi-architecture image that works on both Apple Silicon/ARM chips and Intel/AMD Linux servers, pushing it directly to Docker Hub.

Module 10: Your 4-Week Hands-On Mastery Action Plan

Reading commands is only half the journey—true mastery comes from typing them until they become muscle memory. Follow these four progressive hands-on milestones to go from beginner to confident practitioner:

1. **Milestone 1 (Days 1–5) — Tame Pre-Built Containers:** Pull **`nginx:alpine`** and run it on port **`8080`**. Use **`docker exec -it`** to go inside the running container, locate **`/usr/share/nginx/html/index.html`**, replace the welcome text, and refresh your browser. Then clean up the container and image using **`docker stop`**, **`docker rm`**, and **`docker rmi`**.
2. **Milestone 2 (Days 6–12) — Prove Data Persistence & Networking:** Create a custom network (**`docker network create lab-net`**) and a named volume (**`docker volume create db-data`**). Launch a PostgreSQL container attached to both. Connect to it with **`docker exec -it`**, create a table with sample rows, forcefully delete the container (**`docker rm -f`**), and launch a fresh PostgreSQL container attached to **`db-data`** to verify your table is still there.
3. **Milestone 3 (Days 13–20) — Containerize Your Own App:** Write a small Python (FastAPI/Flask) or Node.js (Express) web server. Write a **`Dockerfile`** and **`.dockerignore`** for it, ordering instructions so dependency installation is cached. Add a non-root **`USER`** and a **`HEALTHCHECK`**, build the image with **`docker build -t myapp:1.0 .`**, and push it to your free Docker Hub account.
4. **Milestone 4 (Days 21–28) — Full-Stack Orchestration with Docker Compose:** Combine your custom web API image, PostgreSQL (with a persistent volume and healthcheck), and Redis into a single **`compose.yaml`** file. Configure a bind mount for live code reloading during development, launch the entire stack with **`docker compose up -d`**, inspect logs with **`docker compose logs -f`**, and tear it down cleanly with **`docker compose down`**.